06 · Control Technical Verification Field Guide

A controls-only deep dive for **highly-technical security specialists**. For each of the **69 SHIELD-AI controls** it gives the identifier, the exact normative text that must hold, the executable policy that verifies it, and the catalogue incidents that evidence the control in practice. This is a working assessment document, not a stakeholder briefing.

Control technical sheet
Control technical sheet

How this sheet is derived

Every control is verified by an executable Rego policy. The sheet above shows, for all 69 controls, the plain objective, the verification depth (deny rules), the exact policy file, and the incidents evidencing the control class. The ‘What to verify’ checklist beneath each control is derived directly from those deny rules.

How to use this field guide

- **Control** is the stable identifier from the framework.
- **What to verify** is the enriched block for a technical specialist: it states the objective, adds the control’s plain purpose (from the policy’s own comment), names the **input model** the executable policy evaluates (so you know what to instrument and feed it), lists each fail-closed condition the policy actually checks, expands each with the concrete tool/interface set it references, and gives the **L3 maturity target** from the policy.
- **Executable policy** names the exact `policies/*.rego` file; open it for the full `deny` rules and the exact input schema.
- **Incident evidence** links each incident to its human-readable summary card in 07 · Incident Summaries, which gives what happened in one sentence and then links

the full catalogue detail in the gap-analysis — this keeps the control page scannable and the deep dive one click further out.

> **How to read the checks:** each bullet is a condition that, when true, makes the control *fail closed* (the action is blocked). To satisfy the control, confirm none of those conditions holds in your deployment, and that `make test` passes. `<name>` marks the runtime subject (agent, tool, server, or record) named in the actual deny message — substitute the real identifier when assessing a deployment.

EE — Execution Environment

EE-01: Execution environment boundary enforcement

- **Executable policy:** `shield_ai_ee_ee-01.rego` (package `shield_ai.ee.execution_environment_boundary`, 1 deny rules)

What to verify: Execution environment boundary enforcement must hold. Enforce a hard execution boundary around every agent. The policy evaluates `input.action.tool, input.agent.sandboxed (bool).` and fails closed – blocks the action – when any of the following is true: - tool is high-risk and agent is not sandboxed (fail closed)

- **Maturity target (L3):** No shared host mounts or namespaces; immutable infrastructure; fail-closed on breach.
- **Incident evidence (14):** INC-11, INC-13, INC-17, INC-18, INC-23, INC-29, INC-38, INC-57, INC-58, INC-59, INC-68, INC-82, INC-91, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-02: Infrastructure API access control

- **Executable policy:** `shield_ai_ee_ee-02.rego` (package `shield_ai.ee.infrastructure_api_access_cont`, 1 deny rules)

What to verify: Infrastructure API access control must hold. Ensure agents cannot reach infrastructure control planes implicitly. The policy evaluates `input.action.tool = the API being called;` and fails closed – blocks the

action – when any of the following is true: - agent call to infrastructure API without explicit tool registration *(relevant set: ec2, iam, lambda, s3, kms, cloudformation, azure_vm, azure_rbac, azure_resource_manager, gcp_compute, gcp_iam, gcp_cloud_resource_manager)*

- **Maturity target (L3):** Explicit tool registration required for infrastructure access; metadata service disabled; blocked attempts logged.
- **Incident evidence (3):** INC-39, INC-60, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-03: Resource usage limiting

- **Executable policy:** `shield_ai_ee_ee-03.rego` (package `shield_ai.ee.resource_usage_limiting`, 3 deny rules)

What to verify: Resource usage limiting must hold. Bound agent resource consumption with runtime enforcement. The policy evaluates `input.agent.resource_usage = {"cpu_pct": n, "network_mbps": n,` and fails closed – blocks the action – when any of the following is true: - agent CPU usage …% exceeds limit …% - agent network usage … Mbps exceeds limit … Mbps - agent API rate … exceeds limit …

- **Maturity target (L3):** Runtime-enforced limits (CPU, memory, storage, network, API rate); exceeding limits alerts and may terminate.
- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-04: Evaluation sandbox integrity

- **Executable policy:** `shield_ai_ee_ee-04.rego` (package `shield_ai.ee.evaluation_sandbox_integrity`, 3 deny rules)

What to verify: Evaluation sandbox integrity must hold. Contain the blast radius of model behaviour during evaluation. The policy evaluates `input.action.kind == "evaluation_run";` and fails closed – blocks the action – when any of the following

is true: - evaluation run with unverified evaluation-to-production isolation (fail closed) - evaluation run has a network path to production - evaluation pipeline can modify production systems or data

- **Maturity target (L3):** No network path between evaluation and production; evaluation pipelines cannot modify production.
- **Incident evidence (3):** INC-19, INC-90, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-05: Host-trusted configuration file injection prevention

- **Executable policy:** `shield_ai_ee_ee-05.rego` (package `shield_ai.ee.host_trusted_configuration_fil`, 2 deny rules)

What to verify: Host-trusted configuration file injection prevention must hold. Prevent indirect sandbox escape via host-trusted file injection. The policy evaluates `input.agent.config.path_writable = paths the agent can write.` and fails closed – blocks the action – when any of the following is true: - agent can write host-trusted path (indirect escape class) *(relevant set: .zshrc, .bashrc, .profile, .vscode/settings.json, .vscode/tasks.json, .git/config, .gitattributes, bin/python, node_modules/.bin, crontab, /etc/systemd/)* - agent file write with no reported write-path allowlist (fail closed)

- **Maturity target (L3):** Host-trusted paths excluded from agent writes; FIM on host-trusted directories; content scanning.
- **Incident evidence (10):** INC-03, INC-11, INC-14, INC-15, INC-41, INC-57, INC-65, INC-76, INC-84, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-06: Fail-closed sandbox operation

- **Executable policy:** `shield_ai_ee_ee-06.rego` (package `shield_ai.ee.fail_closed_sandbox_operation`, 3 deny rules)

What to verify: Fail-closed sandbox operation must hold. Eliminate silent fallback to weaker sandboxes. The policy evaluates `input.action.kind == "code_execution";` and fails closed – blocks the action – when any of the following is true: - interpreter isolation flags are a convenience feature, not a security boundary - no sandbox attestation before code execution by agent (fail closed) - sandbox attestation negative for agent

- **Maturity target (L3):** Runtime attestation before each execution; no fallback path; fail-closed on unavailability; alerts on degradation.
- **Incident evidence (3):** INC-21, INC-61, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-07: Agent control-plane configuration integrity

- **Executable policy:** `shield_ai_ee_ee-07.rego` (package `shield_ai.ee.agent_control_plane_configurat`, 2 deny rules)

What to verify: Agent control-plane configuration integrity must hold. Prevent agents from disabling their own security controls. The policy evaluates `input.agent.config = {"path_writable": [..],` and fails closed – blocks the action – when any of the following is true: - agent configuration loaded from a world-writable location (config-trust class) - agent control-plane path is writable by the agent *(relevant set: .codex/, .claude/, .cursor/, managed-settings.json, mcp.json, .mcp.json, settings.json)*

- **Maturity target (L3):** Control-plane config read-only to the agent; world-writable paths rejected; integrity monitoring on control files.
- **Incident evidence (12):** INC-03, INC-15, INC-20, INC-39, INC-46, INC-53, INC-56, INC-69, INC-75, INC-78, INC-79, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

EE-08: Evaluation-harness isolation and integrity

- **Executable policy:** `shield_ai_ee_ee-08.rego` (package `shield_ai.ee.evaluation_harness_isolation_a`, 3 deny rules)

What to verify: Evaluation-harness isolation and integrity must hold. Ensure evaluation harnesses are genuinely isolated and monitored, with isolation verified rather than asserted. The policy evaluates `input.action.kind == "evaluation_run";` and fails closed – blocks the action – when any of the following is true: - evaluation run without a verified egress block (isolation asserted, not enforced) - evaluation run without target collision-checking against real domains and assets - evaluation run without cross-run separation (contamination risk)

- **Maturity target (L3):** Policy-as-code isolation attestation; GA classifiers and monitoring in eval environments; automated collision checks.
- **Incident evidence (10):** INC-26, INC-27, INC-28, INC-32, INC-40, INC-62, INC-63, INC-64, INC-90, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS — Model Security

MS-01: Model weight protection

- **Executable policy:** `shield_ai_ms_ms-01.rego` (package `shield_ai.ms.model_weight_protection`, 2 deny rules)

What to verify: Model weight protection must hold. Ensure model weights are protected against exfiltration, cloning, and inversion attacks through encryption, access control, and integrity verification. The policy evaluates `input.model = {"weights_encrypted": bool,` and fails closed – blocks the action – when any of the following is true: - model weights not encrypted at rest and in transit - model weights lack key management (HSM or cloud KMS required)

- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-02: Training data integrity

- **Executable policy:** `shield_ai_ms_ms-02.rego` (package `shield_ai.ms.training_data_integrity`, 3 deny rules)

What to verify: Training data integrity must hold. Ensure training and fine-tuning datasets are verified for integrity, provenance, and freedom from poisoning or corruption before use. The policy evaluates `input.training = {"provenance_tracked": bool, "sanitised": bool,` and fails closed – blocks the action – when any of the following is true: - training run with untracked dataset provenance - training run with unsanitised dataset (PII or sensitive content) - training run without a clean poison-detection scan (fail closed)

- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-03: Supply chain security

- **Executable policy:** `shield_ai_ms_ms-03.rego` (package `shield_ai.ms.supply_chain_security`, 3 deny rules)

What to verify: Supply chain security must hold. Verify the provenance and integrity of model weights, ML libraries, training infrastructure, and deployment tooling throughout the AI supply chain. The policy evaluates `input.model.supply_chain = {"sbom": bool, "signature_verified": bool,` and fails closed – blocks the action – when any of the following is true: - model deployed without an ML component SBOM - model provenance signature not verified - model checkpoints not signed

- **Incident evidence (3):** INC-01, INC-19, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-04: Adversarial robustness

- **Executable policy:** `shield_ai_ms_ms-04.rego` (package `shield_ai.ms.adversarial_robustness`, 1 deny rules)

What to verify: Adversarial robustness must hold. Test and verify that models resist prompt injection, jailbreaking, and other adversarial inputs through automated testing, red-teaming, and proxy-layer guardrails. The policy evaluates `input.context.content[] = agent-visible content the action is` and fails closed – blocks the action – when any of the following is true: - prompt-injection or imperceptible-content marker found in agent-visible content

- **Maturity target (L3):** Automated adversarial test suite in CI/CD. Quarterly red-teaming. Output guardrails at proxy layer.
- **Incident evidence (19):** INC-03, INC-08, INC-09, INC-12, INC-13, INC-14, INC-17, INC-20, INC-22, INC-27, INC-29, INC-30, INC-33, INC-42, INC-44, INC-51, INC-54, INC-71, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-05: Model behaviour validation

- **Executable policy:** `shield_ai_ms_ms-05.rego` (package `shield_ai.ms.model_behaviour_validation`, 2 deny rules)

What to verify: Model behaviour validation must hold. Validate that model outputs conform to expected safety, content, and scope boundaries through proxy-layer guardrails and per-agent behaviour configuration. The policy evaluates `input.action.guardrail_evaluated (bool); input.action.scope_compliant` and fails closed – blocks the action – when any of the following is true: - output by agent not evaluated by proxy-layer guardrails (fail closed) - output by agent outside its configured behaviour scope

- **Incident evidence (4):** INC-06, INC-08, INC-09, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-06: Model inventory and lineage tracking

- **Executable policy:** `shield_ai_ms_ms-06.rego` (package `shield_ai.ms.model_inventory_and_lineage_tr`, 3 deny rules)

What to verify: Model inventory and lineage tracking must hold. Maintain a complete, versioned inventory of all models with traceable lineage from training through deployment to retirement. The policy evaluates `input.model.inventory = {"registered": bool, "lineage_tracked": bool,` and fails closed – blocks the action – when any of the following is true: - model not registered in the model inventory - model has no traceable lineage - model has no version record

- **Incident evidence (1):** INC-06 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-07: Deployment context obfuscation

- **Executable policy:** `shield_ai_ms_ms-07.rego` (package `shield_ai.ms.deployment_context_obfuscation`, 2 deny rules)

What to verify: Deployment context obfuscation must hold. Prevent models from detecting evaluation environments and behaving differently than in production. The policy evaluates `input.evaluation = {"env_indistinguishable": bool,` and fails closed – blocks the action – when any of the following is true: - evaluation environment distinguishable from production (endpoints, data shapes, latency) - evaluation behaviour diverges from production for the same inputs (subversion indicator)

- **Incident evidence (2):** INC-19, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-08: MCP

- **Executable policy:** `shield_ai_ms_ms-08.rego` (package `shield_ai.ms.mcp_security_controls`, 3 deny rules)

What to verify: MCP must hold. Secure the MCP protocol layer for agent-tool communication. The policy evaluates `input.action.kind == "mcp_connect";` and fails closed – blocks the action – when any of the following is true: - MCP tool definitions from are not cryptographically signed - MCP server is not from a trusted registry - MCP connection can write execution-bearing locations (registry launch-command class)

- **Maturity target (L3):** Signed tool metadata; response validation against known-good schemas; trusted MCP registry.
- **Incident evidence (19):** INC-12, INC-20, INC-37, INC-39, INC-43, INC-54, INC-56, INC-60, INC-61, INC-76, INC-78, INC-79, INC-80, INC-85, INC-86, INC-87, INC-88, INC-90, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-09: Perception-fidelity enforcement for human-in-the-loop approvals

- **Executable policy:** `shield_ai_ms_ms-09.rego` (package `shield_ai.ms.perception_fidelity_enforcemen`, 2 deny rules)

What to verify: Perception-fidelity enforcement for human-in-the-loop approvals must hold. Ensure human-in-the-loop approval is never gated on content the human cannot perceive. The policy evaluates `input.approval = {"present": bool, "snapshot_fidelity_verified":` and fails closed – blocks the action – when any of the following is true: - approval evidence comes from an agent-generated attestation (sockpuppet class); out-of-band verification required - approval binds to content containing imperceptible marker without a verified rendering-fidelity snapshot *(relevant set: color:#fff;font-size:1px, font-size:0px, opacity:0, opacity: 0, display:none, visibility:hidden, white-space:pre-wrap;color:transparent)*

- **Maturity target (L3):** Policy-as-code fidelity checks pre-approval; multimodal sanitisation; hidden-content stripping before agent context.
- **Incident evidence (10):** INC-25, INC-27, INC-30, INC-36, INC-41, INC-55, INC-71, INC-72, INC-84, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-10: Agent memory-store integrity

- **Executable policy:** `shield_ai_ms_ms-10.rego` (package `shield_ai.ms.agent_memory_store_integrity`, 2 deny rules)

What to verify: Agent memory-store integrity must hold. Ensure agent persistent memory cannot be silently poisoned to steer future behaviour. The policy evaluates `input.action.kind == "memory_write";` and fails closed – blocks the action – when any of the following is true: - memory-store write by agent is unauthenticated - memory-store write by agent carries tamper evidence; store must be treated as compromised

- **Maturity target (L3):** Tamper-evident memory (append-only or immutable snapshots); anomaly detection on memory writes via MIR-02; retrieval-layer filtering of flagged entries; documented post-incident memory audit-and-clean procedure.
- **Incident evidence (7):** INC-31, INC-33, INC-45, INC-70, INC-81, INC-91, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-11: Operational data-stream integrity

- **Executable policy:** `shield_ai_ms_ms-11.rego` (package `shield_ai.ms.operational_data_stream_integr`, 3 deny rules)

What to verify: Operational data-stream integrity must hold. Prevent poisoned logs, alerts, and observability feeds from commanding the agent. The policy evaluates `input.action.trigger == "log_alert" when the action's proximate` and fails closed – blocks the action – when any of the following is true: - agent action triggered by log/alert content from an unverified producer - instruction-bearing operational content acted on without independent non-log verification - high-impact agent action triggered by log/alert content without human approval

- **Maturity target (L3):** Agent-visible operational content sanitised for instruction-bearing payloads before reasoning; independent non-log verification for log-

triggered actions; ingestion pipelines authenticate producers; MIR-02 flags log-origin agent actions.

- **Incident evidence (4):** INC-33, INC-40, INC-67, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-12: MCP discovery-metadata and shared-cache integrity

- **Executable policy:** `shield_ai_ms_ms-12.rego` (package `shield_ai.ms.mcp_discovery_metadata_shared_cache_integrity`, 6 deny rules)

What to verify: MCP discovery-metadata and shared-cache integrity must hold. Prevent MCP discovery metadata and shared caches from injecting instructions into agent system prompts. The executable policy fails closed – blocks the action – when any of the following is true: - MCP discovery response for agent carries unbounded server-supplied instructions; must be length-limited - MCP discovery response for agent folds server-supplied instructions into the trusted system prompt; must be isolated as untrusted content - MCP discovery response for agent is served from a shared cache across authentication contexts without signed provenance - MCP discovery response for agent trusts a server cacheScope claim without verification - MCP discovery response for agent uses a cache key not bound to the authenticated identity - *1 more fail-closed conditions in the policy.*

- **Maturity target (L3):** Instructions isolated from the trusted system prompt with explicit boundaries and length limits; cache keys bind to authenticated identity; cacheScope claims verified; tool metadata pinned to a baseline with drift alerting.
- **Incident evidence (3):** INC-37, INC-43, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-13: Transcript and conversation authenticity

- **Executable policy:** `shield_ai_ms_ms-13.rego` (package `shield_ai.ms.transcript_and_conversation_authenticity`, 4 deny rules)

What to verify: Transcript and conversation authenticity must hold. Make the harness's record of model output tamper-evident and authentic at the model boundary. The executable policy fails closed – blocks the action – when any of the following is true: - agent reads a transcript whose model outputs are not signed at the model boundary; stored response cannot be verified as genuine - agent proceeds from an unverified transcript; every stored response must be verified on read before reasoning - agent attempts a confidence-gated action from a transcript with a broken authenticity chain; chain-break blocks execution - agent reads from an unverified transcript store; transcript stores are untrusted until verified

- **Maturity target (L3):** Model outputs signed at the API/proxy boundary (per-session HMAC/Ed25519, key outside harness) and verified on load; verification failures alert via MIR-01 and block the action.
- **Incident evidence (3):** INC-70, INC-83, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-14: Self-replication barrier

- **Executable policy:** `shield_ai_ms_ms-14.rego` (package `shield_ai.ms.self_replication_barrier`, 4 deny rules)

What to verify: Self-replication barrier must hold. Prevent an injected instruction from being re-emitted through the agent's own output channels to propagate to other agents. The executable policy fails closed – blocks the action – when any of the following is true: - outbound publish by agent matches an instruction-bearing n-gram fingerprint (replication vector) and is not quarantined - outbound publish by agent is a fingerprinted replication match that lacks human approval - outbound broadcast by agent lacks human signoff (one-channel-per-write / human-signoff-on-broadcast) - outbound publish by agent carries a propagation signal; channel-kill and incident response triggered (worm-emulation)

- **Maturity target (L3):** Fingerprint matching quarantine with human approval; one-channel-per-write and human-signoff-on-broadcast; blast-radius cap per hop.
- **Incident evidence (2):** INC-71, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MS-15: Cross-tenant agent memory and state isolation

- **Executable policy:** `shield_ai_ms_ms-15.rego` (package `shield_ai.ms.cross_tenant_agent_memory_and_state`, 8 deny rules)

What to verify: Cross-tenant agent memory and state isolation must hold. Isolate multi-tenant agent memory and state per tenant as a hard boundary; keep the isolation key out of model control and gate retrieval/write by verified ownership. The executable policy fails closed – blocks the action – when any of the following is true: - pooled memory_read by agent returned a cross-tenant hit (owner != querying principal); hard ownership gate must drop it - pooled memory_read by agent could not verify ownership of a hit; fail closed per default-deny (semantic similarity is not an admissibility proxy) - cross-tenant memory_write by agent (attempted write outside the writing principal's tenant); denied and alarmed - memory_write by agent is not authenticated to a verified writer identity - memory_read by agent uses an LLM-controllable namespace/tenant key; the isolation key must never be a model-controllable tool argument - *3 more fail-closed conditions in the policy.*

- **Maturity target (L3):** Hard post-retrieval and post-write ownership gating with default-deny; connection parameters bound at construction; cross-tenant hits alarmed as control violations.
- **Incident evidence (4):** INC-81, INC-87, INC-91, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM — Identity & Access

IAM-01: Agent identity management

- **Executable policy:** `shield_ai_iam_iam-01.rego` (package `shield_ai.iam.agent_identity_management`, 2 deny rules)

What to verify: Agent identity management must hold. Ensure every agent has a unique, lifecycle-managed digital identity distinct from human user identities,

integrated with the organisation's identity provider. The policy evaluates `input.agent.identity = {"unique": bool, "auditable": bool}.` and fails closed – blocks the action – when any of the following is true: - agent has no unique identity (sharing human credentials) - agent identity is not auditable

- **Incident evidence (5):** INC-04, INC-16, INC-35, INC-52, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-02: Scope-bound delegation

- **Executable policy:** `shield_ai_iam_iam-02.rego` (package `shield_ai.iam.scope_bound_delegation`, 2 deny rules)

What to verify: Scope-bound delegation must hold. Limit each agent's permissions to the minimum set of tools, data, and actions required for its function, enforced by a policy-as-code engine at invocation time. The policy evaluates `input.agent.delegation = {"scoped": bool, "wildcard": bool}.` and fails closed – blocks the action – when any of the following is true: - agent has no scope-bound delegation claim (fail closed) - agent has wildcard permissions

- **Incident evidence (10):** INC-04, INC-05, INC-09, INC-10, INC-15, INC-16, INC-42, INC-47, INC-55, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-03: Human oversight controls

- **Executable policy:** `shield_ai_iam_iam-03.rego` (package `shield_ai.iam.human_oversight_controls`, 3 deny rules)

What to verify: Human oversight controls must hold. Implement risk-tiered human approval gates for agent actions, with documented break-glass procedures for urgent situations. The policy evaluates `input.action.high_risk_class ("financial" | "data_write" |` and fails closed – blocks the action – when any of the following is true: - approval evidence from session history is not human approval (forged-

approval class) - financial action approved on the agent channel; human-console approval required - … action by agent without human approval

- **Maturity target (L3):** Risk-based approval tiers. Automated approval gates. Time-limited approvals. Break-glass procedures documented.
- **Incident evidence (8):** INC-10, INC-20, INC-21, INC-23, INC-27, INC-34, INC-55, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-04: Credential isolation

- **Executable policy:** `shield_ai_iam_iam-04.rego` (package `shield_ai.iam.credential_isolation`, 2 deny rules)

What to verify: Credential isolation must hold. Store, manage, and rotate agent credentials separately from human credentials with hardware-backed protection for critical agents and automated rotation on compromise signals. The policy evaluates `input.agent.credentials = {"isolated": bool, "hardware_backed":` and fails closed – blocks the action – when any of the following is true: - agent credentials not isolated from human credentials - agent compromise signal without credential rotation

- **Incident evidence (4):** INC-01, INC-04, INC-19, INC-21 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-05: Just-in-time access

- **Executable policy:** `shield_ai_iam_iam-05.rego` (package `shield_ai.iam.just_in_time_jit_access`, 2 deny rules)

What to verify: Just-in-time access must hold. Issue session-bound, ephemeral credentials for agents with automatic expiration and the ability to revoke mid-session on suspicious behaviour detection. The policy evaluates `input.agent.credentials = {"ephemeral": bool,` and fails closed – blocks the action – when any of the

following is true: - agent has standing (non-ephemeral) credentials - agent session expired without credential revocation

- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-06: Cross-session boundary enforcement

- **Executable policy:** `shield_ai_iam_iam-06.rego` (package `shield_ai.iam.cross_session_boundary_enforce`, 2 deny rules)

What to verify: Cross-session boundary enforcement must hold. Ensure each agent session is an independent security context with no credential or data sharing across sessions. The policy evaluates `input.agent.session = {"independent_context": bool,` and fails closed – blocks the action – when any of the following is true: - agent session shares credentials across sessions - agent session is not an independent security context

- **Incident evidence (2):** INC-50, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-07: Transitive delegation chain enforcement

- **Executable policy:** `shield_ai_iam_iam-07.rego` (package `shield_ai.iam.transitive_delegation_chain_en`, 2 deny rules)

What to verify: Transitive delegation chain enforcement must hold. Bound delegation chains and make them fully auditable. The policy evaluates `input.agent.delegation = {"hops": n, "max_hops": 2,` and fails closed – blocks the action – when any of the following is true: - delegation chain for agent is … hops, exceeding the bound of … - delegation chain for agent is not reconstructable from audit logs

- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-08: Credential scope narrowing

- **Executable policy:** `shield_ai_iam_iam-08.rego` (package `shield_ai.iam.credential_scope_narrowing`, 3 deny rules)

What to verify: Credential scope narrowing must hold. Ensure credentials in agent environments are minimal, short-lived, and revocable. The policy evaluates `input.agent.credentials = {"per_tool_scope": bool,` and fails closed – blocks the action – when any of the following is true: - agent holds a broad service role instead of per-tool scoped credentials - agent credentials are not scoped per registered tool - agent credentials are not short-lived

- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-09: Multi-tenant agent platform isolation

- **Executable policy:** `shield_ai_iam_iam-09.rego` (package `shield_ai.iam.multi_tenant_agent_platform_is`, 2 deny rules)

What to verify: Multi-tenant agent platform isolation must hold. Enforce data-layer tenant isolation on managed agent platforms. The policy evaluates `input.tenant = {"id": string, "scoped": bool};` and fails closed – blocks the action – when any of the following is true: - request not scoped to tenant ID at the data layer (presentation-layer checks only) - user-controlled object ID bypasses tenant scoping (CWE-639 IDOR class)

- **Incident evidence (4):** INC-23, INC-35, INC-52, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

IAM-10: Taint-tracked least-privilege subagent scoping

- **Executable policy:** `shield_ai_iam_iam-10.rego` (package `shield_ai.iam.taint_tracked_least_privilege_subagent_scoping`, 4 deny rules)

What to verify: Taint-tracked least-privilege subagent scoping must hold. Taint-track externally-submitted records and bind subagent permission scoping to that taint plus field-level least privilege. The policy evaluates `input.action.kind == "data_read"; input.action.record.tainted =` and fails closed – blocks the action – when any of the following is true: - agent processed an externally-submitted record that was not taint-marked at ingestion - agent reads a tainted record without field-level least-privilege scoping (must be narrower than nominal grants) - agent tainted read cascades into an untainted sensitive collection beyond the immediate task; blocked - agent processed a tainted record where the untrusted/trusted sanitisation boundary is not an explicit access-control decision

- **Maturity target (L3):** Taint-marked at ingestion; record-handling subagents scoped to only required tables/fields/tools; tainted reads prevented from cascading into untainted sensitive collections.
- **Incident evidence (2):** INC-72, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS — Data Security

DS-01: Data classification and labelling

- **Executable policy:** `shield_ai_ds_ds-01.rego` (package `shield_ai.ds.data_classification_and_labell`, 2 deny rules)

What to verify: Data classification and labelling must hold. Classify all data entering the AI system at ingestion using an automated pipeline that feeds into downstream encryption, retention, and access controls. The policy evaluates `input.data = {"classified": bool, "label": string}.` and fails closed – blocks the action –

when any of the following is true: - data ingestion without automated classification (fail closed) - data ingestion with no classification label

- **Incident evidence (3):** INC-02, INC-15, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-02: Inference data protection

- **Executable policy:** `shield_ai_ds_ds-02.rego` (package `shield_ai.ds.inference_data_protection`, 3 deny rules)

What to verify: Inference data protection must hold. Encrypt all data in transit between user, agent, model, and tools with TLS 1.3 minimum, and secure inference endpoints within the organisation's trust boundary for sensitive workloads. The policy evaluates `input.data.inference = {"encrypted_e2e": bool, "tls13_min": bool,` and fails closed – blocks the action – when any of the following is true: - inference data not encrypted end-to-end - inference transport below TLS 1.3 minimum - sensitive inference endpoint outside the organisation's trust boundary

- **Incident evidence (1):** INC-04 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-03: Memory sanitisation

- **Executable policy:** `shield_ai_ds_ds-03.rego` (package `shield_ai.ds.memory_sanitisation`, 2 deny rules)

What to verify: Memory sanitisation must hold. Prevent persistence of sensitive data across sessions through automated purging, encryption, per-agent scoping, and cross-session aggregation detection. The policy evaluates `input.agent.memory = {"persists_classified": bool,` and fails closed – blocks the action – when any of the following is true: - agent persists classified or sensitive data beyond session lifetime - agent session ended without memory wipe (cross-session leakage risk)

- **Incident evidence (2):** INC-19, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-04: Data minimisation

- **Executable policy:** `shield_ai_ds_ds-04.rego` (package `shield_ai.ds.data_minimisation`, 3 deny rules)

What to verify: Data minimisation must hold. Filter data at the retrieval layer (field-level, not document-level) and enforce attribute-based access control at query time to ensure agents receive only the minimum data required. The policy evaluates `input.data.access = {"bulk": bool, "field_filtered": bool,` and fails closed – blocks the action – when any of the following is true: - bulk data access by agent (field-level filtering required) - retrieval without field-level filtering (document-level only) - retrieval without attribute-based access control at query time

- **Incident evidence (2):** INC-23, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-05: Training data governance

- **Executable policy:** `shield_ai_ds_ds-05.rego` (package `shield_ai.ds.training_data_governance`, 3 deny rules)

What to verify: Training data governance must hold. Track training data lineage from source to model, verify consent for personal data, enforce retention limits, and provide machine unlearning capability. The policy evaluates `input.training.governance = {"lineage": bool, "retention_limited":` and fails closed – blocks the action – when any of the following is true: - training run without dataset lineage tracking - training run on personal data without verified consent - training run without retention limits on source datasets

- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-06: Output validation

- **Executable policy:** `shield_ai_ds_ds-06.rego` (package `shield_ai.ds.output_validation`, 2 deny rules)

What to verify: Output validation must hold. Scan all model outputs for data leakage (PII, training data, system prompts, credentials) at the inference proxy layer before delivery to users or systems. The policy evaluates `input.action.output_scan = "clean" | "leak" | "pending".` and fails closed – blocks the action – when any of the following is true: - output not verified clean by the leakage scanner (fail closed) - output flagged for data leakage (PII, training data, system prompt, or credential)

- **Maturity target (L3):** Automated output scanning for PII, sensitive data, system prompts, memorised training data. Scanning at application layer.
- **Incident evidence (6):** INC-02, INC-19, INC-22, INC-51, INC-71, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-07: Agent-originated artifact publication control

- **Executable policy:** `shield_ai_ds_ds-07.rego` (package `shield_ai.ds.agent_originated_artifact_publ`, 3 deny rules)

What to verify: Agent-originated artifact publication control must hold. Prevent agents from unintentionally or maliciously publishing execution-bearing artifacts into the software supply chain. The policy evaluates `input.action.kind == "publication"; input.action.target = the` and fails closed – blocks the action – when any of the following is true: - execution-bearing configuration write requires human-console approval (Kiro class) - agent publication to lacks the required out-of-band approval - publication to was approved on the agent channel; out-of-band human approval required

- **Maturity target (L3):** Signed, scanned, provenance-tracked publication pipeline; short-lived scoped credentials; agent cannot self-approve.
- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

DS-08: Destination and emergent-channel egress inventory

- **Executable policy:** `shield_ai_ds_ds-08.rego` (package `shield_ai.ds.destination_and_emergent_channel_egress_inventory`, 6 deny rules)

What to verify: Destination and emergent-channel egress inventory must hold. Enumerate and control every outbound destination and channel that can carry agent data out of the boundary, using payload-blind allow-lists rather than classifiers. The executable policy fails closed – blocks the action – when any of the following is true: - agent egress to destination not on the payload-blind allow-list - agent egress through an un-inventoried sink (channel not enumerated/mapped to a control) - agent egress rely on a content classifier rather than a payload-blind hard allow-list - agent messaging egress lacks invoker attribution and user confirmation - agent DNS egress is not restricted to allow-listed resolvers with inspected names - *1 more fail-closed conditions in the policy.*

- **Maturity target (L3):** Full channel inventory (URLs, DNS, image/render sources, messaging/unfurl, browser) each mapped to an explicit control; allow-list verification is payload-blind.
- **Incident evidence (2):** INC-72, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP — Tool & Pipeline Security

TP-01: Tool inventory and registry

- **Executable policy:** `shield_ai_tp_tp-01.rego` (package `shield_ai.tp.tool_inventory_and_registry`, 2 deny rules)

What to verify: Tool inventory and registry must hold. Maintain a central registry of all tools accessible to agents, with security review, versioning, and integration with the authorisation system. The policy evaluates `input.action.tool = the tool`

`being invoked. The registry below` and fails closed – blocks the action – when any of the following is true: - tool is not in the approved tool registry

- **Maturity target (L3):** Central tool registry with full metadata. Automated security review. Integration with authorisation.
- **Incident evidence (10):** INC-03, INC-04, INC-05, INC-08, INC-09, INC-10, INC-11, INC-17, INC-44, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-02: Tool access control

- **Executable policy:** `shield_ai_tp_tp-02.rego` (package `shield_ai.tp.tool_access_control`, 3 deny rules)

What to verify: Tool access control must hold. Enforce per-agent tool allow-lists at invocation time with authenticated agent identity and version-controlled, auditable access policies. The policy evaluates `input.action.tool_auth = {"agent_authenticated": bool,` and fails closed – blocks the action – when any of the following is true: - tool call not authenticated to the agent identity - tool authenticated against the human session, not the agent identity - tool not on the agent's invocation-time allow-list

- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-03: API gateway security

- **Executable policy:** `shield_ai_tp_tp-03.rego` (package `shield_ai.tp.api_gateway_security`, 3 deny rules)

What to verify: API gateway security must hold. Deploy a gateway between agents and tools that enforces rate limiting, schema validation, audit logging, and MCP protocol awareness where applicable. The policy evaluates `input.action.gateway = {"rate_limited": bool,` and fails closed – blocks the action – when any of the

following is true: - tool call not rate-limited by the gateway - tool call payload not schema-validated by the gateway - tool call not audit-logged by the gateway

- **Incident evidence (9):** INC-01, INC-04, INC-07, INC-08, INC-12, INC-13, INC-21, INC-23, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-04: Tool output validation

- **Executable policy:** `shield_ai_tp_tp-04.rego` (package `shield_ai.tp.tool_output_validation`, 4 deny rules)

What to verify: Tool output validation must hold. Validate all tool outputs against known-good schemas before passing to agent context, with content-type verification, size limits, and sandboxed execution for code-generation tools. The policy evaluates `input.action.tool_output = {"validated": bool, "sanitised": bool,` and fails closed – blocks the action – when any of the following is true: - tool output not validated against a known-good schema - tool output not sanitised before entering agent context - tool output exceeds the size limit - code-generation output of tool not executed in a sandbox

- **Incident evidence (13):** INC-03, INC-05, INC-09, INC-11, INC-12, INC-13, INC-14, INC-16, INC-17, INC-18, INC-20, INC-22, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-05: Pipeline integrity

- **Executable policy:** `shield_ai_tp_tp-05.rego` (package `shield_ai.tp.pipeline_integrity`, 4 deny rules)

What to verify: Pipeline integrity must hold. Sign and attest all CI/CD pipeline runs, enforce security gates before deployment, and store deployment artifacts in an immutable, integrity-verified registry. The policy evaluates `input.pipeline = {"run_signed": bool, "attested": bool,` and fails closed – blocks the action – when any of the following is true: - deployment from an unsigned CI/CD run - deployment without pipeline attestation - deployment that failed the security

validation gate - deployment artifact not stored in an immutable, integrity-verified registry

- **Incident evidence (8):** INC-01, INC-05, INC-07, INC-19, INC-22, INC-47, INC-55, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-06: Dependency scanning

- **Executable policy:** `shield_ai_tp_tp-06.rego` (package `shield_ai.tp.dependency_scanning`, 3 deny rules)

What to verify: Dependency scanning must hold. Continuously scan all ML libraries, model dependencies, tool SDKs, and container images for vulnerabilities with automated blocking of critical-severity findings. The policy evaluates `input.dependencies = {"scanned": bool, "critical_finding": bool,` and fails closed – blocks the action – when any of the following is true: - deployment with unscanned dependencies - critical-severity dependency finding not blocked - HIGH-RISK sandbox library (vm2/n8n/Pyodide/Wasmtime class) with critical finding and no approved alternative

- **Incident evidence (9):** INC-07, INC-13, INC-18, INC-19, INC-38, INC-44, INC-48, INC-58, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-07: Cross-tool dependency analysis

- **Executable policy:** `shield_ai_tp_tp-07.rego` (package `shield_ai.tp.cross_tool_dependency_analysis`, 2 deny rules)

What to verify: Cross-tool dependency analysis must hold. Detect and gate escape-class tool combinations before deployment. The policy evaluates `input.tools = {"chain_analysed": bool, "escape_chain": bool,` and fails closed – blocks the action – when any of the following is true: - deployment without cross-tool

dependency (transitive risk) analysis - tool combination enables an escape-class chain without explicit risk acceptance (GRM-02)

- **Incident evidence (2):** INC-12, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-08: API gateway infrastructure-isolation mode

- **Executable policy:** `shield_ai_tp_tp-08.rego` (package `shield_ai.tp.api_gateway_infrastructure_iso`, 2 deny rules)

What to verify: API gateway infrastructure-isolation mode must hold. Prevent agents from reaching cloud infrastructure control planes. The policy evaluates `input.action.gateway = {"infra_isolation": bool,` and fails closed – blocks the action – when any of the following is true: - tool call without an active gateway API allow-list - infrastructure control-plane API not blocked by the gateway (infra-isolation mode off)

- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-09: Explicit tool registration — no implicit tool exposure

- **Executable policy:** `shield_ai_tp_tp-09.rego` (package `shield_ai.tp.explicit_tool_registration`, 2 deny rules)

What to verify: Explicit tool registration — no implicit tool exposure must hold. Eliminate implicit tool exposure in agent frameworks. The policy evaluates `input.action.kind in {"tool_call","dispatch"};` and fails closed – blocks the action – when any of the following is true: - tool was not explicitly registered (implicit exposure class) - tool auto-registered by framework mechanism; explicit registration required

- **Maturity target (L3):** Auto-registration disabled by default; all callables audited; unregistered callables blocked.

- **Incident evidence (5):** INC-11, INC-34, INC-46, INC-48, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-10: Agent platform inbound control-plane protection

- **Executable policy:** `shield_ai_tp_tp-10.rego` (package `shield_ai.tp.agent_platform_inbound_control`, 2 deny rules)

What to verify: Agent platform inbound control-plane protection must hold. Defend the agent platform's inbound listening surface from direct, model-independent attack. The policy evaluates `input.action.kind == "binding"; input.action.target = listener;` and fails closed – blocks the action – when any of the following is true: - listener binding is not on the approved binding list (default deny) - web-origin binding without origin validation (CSWSH class)

- **Maturity target (L3):** Default-deny binding policy enforced; no standing allow rules; automated detection of new listeners; no operator-scope tokens readable from web context.
- **Incident evidence (13):** INC-24, INC-30, INC-31, INC-32, INC-34, INC-39, INC-45, INC-54, INC-56, INC-60, INC-61, INC-79, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-11: Agent browser and content-rendering isolation

- **Executable policy:** `shield_ai_tp_tp-11.rego` (package `shield_ai.tp.agent_browser_and_content_rend`, 3 deny rules)

What to verify: Agent browser and content-rendering isolation must hold. Prevent untrusted rendered content from commanding the agent. The policy evaluates `input.action.cross_origin (bool), input.action.privileged (bool),` and fails closed – blocks the action – when any of the following is true: - document upload rendered without sanitisation before assistant processing - token access from

rendered content without approval (credential-layer isolation) - cross-origin privileged action from rendered content without approval

- **Maturity target (L3):** Origin isolation preserved in agent browser runtimes; no cross-origin token access; imperative action gating (approval required for financial and credential operations); document uploads sanitised before assistant processing.
- **Incident evidence (4):** INC-29, INC-30, INC-42, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-12: Tool-call provenance and dispatch verification

- **Executable policy:** `shield_ai_tp_tp-12.rego` (package `shield_ai.tp.tool_call_provenance_and_dispa`, 4 deny rules)

What to verify: Tool-call provenance and dispatch verification must hold. Ensure no tool executes without verified provenance from an authorised model turn or caller. The policy evaluates `input.action.kind == "dispatch"; input.action.provenance =` and fails closed – blocks the action – when any of the following is true: - dispatch without session provenance (fail closed) - dispatch without turn provenance (fail closed) - dispatch without model signature (fail closed) - approval bound to does not match dispatched tool (forged-approval class)

- **Maturity target (L3):** Tool calls carry cryptographically bound provenance (session, turn, model signature) verified before dispatch; session/event history authenticated; approval records bind to tool name and arguments and verify against the human approver channel; fail closed on unverifiable provenance.
- **Incident evidence (8):** INC-11, INC-34, INC-41, INC-44, INC-46, INC-53, INC-84, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-13: Agent dependency-install provenance verification

- **Executable policy:** `shield_ai_tp_tp-13.rego` (package `shield_ai.tp.agent_dependency_install_provenance`, 6 deny rules)

What to verify: Agent dependency-install provenance verification must hold. Verify registry-namespace and domain-ownership provenance before any agent-executed install derived from ingested content. The policy evaluates `input.action.kind == "dependency_install". Installation commands` and fails closed – blocks the action – when any of the following is true: - install by agent references a package not confirmed to exist in the intended registry - install by agent references a registry namespace not verified to belong to the documented vendor - install by agent references an unclaimed or expired domain; dangling reference must block the install - install by agent is derived from ingested content and is not gated by an allowlist or human approval - install by agent treats a documentation file as an authoritative install script; a doc file is never the install script - *1 more fail-closed conditions in the policy.*

- **Maturity target (L3):** Pre-install provenance verification: registry existence, namespace ownership, domain registration; installs from untrusted content gated by an allowlist or human approval; dangling references blocked and logged via MIR-01.
- **Incident evidence (7):** INC-44, INC-48, INC-66, INC-80, INC-85, INC-86, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-14: Skill/plugin payload integrity and instruction-channel isolation

- **Executable policy:** `shield_ai_tp_tp-14.rego` (package `shield_ai.tp.skill_plugin_payload_integrity`, 6 deny rules)

What to verify: Skill/plugin payload integrity and instruction-channel isolation must hold. Gate every agent extension install on signed, immutable payloads with no runtime-fetchable instruction channels and privilege-scoped execution. The policy evaluates `input.action.kind == "skill_execute" (a skill/plugin about to run).` and fails closed – blocks the action – when any of the following is true: - execute skill for agent with an unsigned payload; never execute unsigned - execute skill for agent whose payload is not immutable at install time - skill for agent has a runtime-fetchable instruction channel (post-install reference/update feed); must be blocked at network and runtime - skill for agent executes with full agent privileges rather than a

declared capability scope - skill for agent lacks end-to-end provenance attestation (publisher -> registry -> artifact) - *1 more fail-closed conditions in the policy.*

- **Maturity target (L3):** Immutable signed payloads; runtime-fetch channels blocked; privilege-scoped execution; post-install baseline with drift quarantine.
- **Incident evidence (3):** INC-48, INC-66, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-15: In-process code-execution sandbox sufficiency

- **Executable policy:** `shield_ai_tp_tp-15.rego` (package `shield_ai.tp.in_process_code_execution_sandbox_sufficiency`, 5 deny rules)

What to verify: In-process code-execution sandbox sufficiency must hold. Route all LLM-generated or untrusted code execution to an OS/VM-isolated execution environment; never rely on an in-process denylist/blocklist interpreter gate as the boundary. The policy evaluates `input.action.kind == "code_execution". LLM-generated or untrusted` and fails closed – blocks the action – when any of the following is true: - code execution by agent runs LLM-generated/untrusted code outside an OS/VM-isolated boundary - code execution by agent relies on an in-process denylist/blocklist gate as the sole containment; structurally bypassable - code execution by agent separates trust by an in-process context that shares a memory heap; not a security boundary (Codex Heapjack class) - code execution by agent uses an in-process sandbox not treated as the HIGH-RISK dependency class with blocking on critical findings - code execution by agent runs a chat-template/ rendering-template engine as a code surface without EE-01-grade render isolation (CPython/Jinja template-escape class)

- **Maturity target (L3):** In-process sandboxes locked in the HIGH-RISK dependency class (TP-06) with automated blocking on critical findings; model-generated code executes only in the EE sandbox.
- **Incident evidence (11):** INC-58, INC-65, INC-68, INC-74, INC-76, INC-77, INC-80, INC-85, INC-87, INC-91, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

TP-16: Tool-connection and launch-path integrity

- **Executable policy:** `shield_ai_tp_tp-16.rego` (package `shield_ai.tp.tool_connection_and_launch_path_integrity`, 6 deny rules)

What to verify: Tool-connection and launch-path integrity must hold. Keep remote peer-supplied protocol metadata (OAuth/discovery/connection fields) out of process-launch and shell execution primitives; origin-bound and validate the authorization endpoint. The policy evaluates `input.action.kind == "tool_connect" / "tool_launch". Remote` and fails closed – blocks the action – when any of the following is true: - agent connection hands server-supplied protocol metadata to a process-launch/shell primitive (code-execution boundary) - agent authorization endpoint is not origin-bound to the configured server - agent accepts the server-advertised authorization endpoint without validating the canonical processed value against the configured origin - agent launch command registered through a tool/server definition is not on the allowlist - agent auto-launch command is model-influenced or externally derived; a server definition must not register an arbitrary OS command - *1 more fail-closed conditions in the policy.*

- **Maturity target (L3):** Authorization endpoint origin-bound and validated on canonical value; no transport field crosses into a launch/shell primitive; launch commands allowlisted; connection termination and alert on any violation.
- **Incident evidence (6):** INC-79, INC-85, INC-86, INC-88, INC-89, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MIR — Monitoring, Logging & IR

MIR-01: Agent activity logging

- **Executable policy:** `shield_ai_mir_mir-01.rego` (package `shield_ai.mir.agent_activity_logging`, 2 deny rules)

What to verify: Agent activity logging must hold. Log all agent actions in structured format with immutable storage, SIEM integration, and distributed tracing support for multi-agent architectures. The policy evaluates `input.action.audit_record (bool) -- the action carries a` and fails closed – blocks the action – when any of the following is true: - audit record not written to append-only storage (integrity requirement) - agent action has no structured audit record

- **Maturity target (L3):** Immutable logs. Correlation IDs across multi-agent chains. SIEM integration. Log integrity verification.
- **Incident evidence (10):** INC-01, INC-05, INC-06, INC-08, INC-12, INC-16, INC-19, INC-48, INC-56, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MIR-02: Anomaly detection

- **Executable policy:** `shield_ai_mir_mir-02.rego` (package `shield_ai.mir.behavioural_anomaly_detection`, 2 deny rules)

What to verify: Anomaly detection must hold. Establish per-agent and per-agent-class behaviour baselines for tool usage, data access, memory access, and decision patterns, with automated alerting on deviation. The policy evaluates `input.action.anomaly_flag (bool) -- the platform's baseline` and fails closed – blocks the action – when any of the following is true: - action by agent deviates from its behaviour baseline - sensitive tool from ephemeral sandbox agent without campaign correlation (swarm class) *(relevant set: transfer_funds, write_file, shell_exec, send_email_draft, delete_objects)*

- **Maturity target (L3):** Per-agent and per-class baselines. Automated alerting. Integration with SOC. Regular tuning.
- **Incident evidence (18):** INC-06, INC-08, INC-10, INC-19, INC-23, INC-27, INC-28, INC-32, INC-35, INC-40, INC-43, INC-45, INC-49, INC-55, INC-56, INC-63, INC-90, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MIR-03: Prompt audit log

- **Executable policy:** `shield_ai_mir_mir-03.rego` (package `shield_ai.mir.prompt_audit_log`, 2 deny rules)

What to verify: Prompt audit log must hold. Log prompt/response pairs with automated sensitive data redaction, tiered retention, and SIEM integration for forensic analysis. The policy evaluates `input.action.sensitive (bool) -- the action involves sensitive` and fails closed – blocks the action – when any of the following is true: - sensitive prompt/response pair not captured for the audit log - captured prompt/response pair not redacted before SIEM ingestion

- **Maturity target (L3):** Automated redaction. Tiered retention (full -> redacted). SIEM integration.
- **Incident evidence (1):** INC-22 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MIR-04: Incident response

- **Executable policy:** `shield_ai_mir_mir-04.rego` (package `shield_ai.mir.incident_response_playbooks`, 2 deny rules)

What to verify: Incident response must hold. Develop and maintain AI-specific IR playbooks covering prompt injection, model compromise, tool abuse, and code-generation attacks, tested through regular tabletop exercises. The policy evaluates `input.incident = {"playbook": string, "containment": bool,` and fails closed – blocks the action – when any of the following is true: - … incident acknowledged without a named AI-specific IR playbook - … incident without an automatic containment step (agent-speed response)

- **Incident evidence (3):** INC-07, INC-16, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MIR-05: Mandatory reporting integration

- **Executable policy:** `shield_ai_mir_mir-05.rego` (package `shield_ai.mir.mandatory_reporting_integratio`, 1 deny rules)

What to verify: Mandatory reporting integration must hold. Integrate incident detection with automated regulatory reporting triggers for SOCI Act, APRA CPS 234, and Notifiable Data Breaches, with pre-populated report templates. The policy evaluates `input.action.incident (bool); input.action.reporting_triggered` and fails closed – blocks the action – when any of the following is true: - incident reported by agent has no regulatory reporting trigger (SOCI 12h / APRA 72h windows)

- **Maturity target (L3):** Automated incident detection triggers pre-populated reports. Escalation workflows active.
- **Incident evidence:** none yet — baseline coverage, not yet demonstrated by a catalogue incident

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

MIR-06: Forensic readiness

- **Executable policy:** `shield_ai_mir_mir-06.rego` (package `shield_ai.mir.forensic_readiness`, 1 deny rules)

What to verify: Forensic readiness must hold. Maintain immutable, verifiable logs with full session reconstruction capability for all agents, including support for multi-agent chain reconstruction. The policy evaluates `input.action.high_impact (bool); input.action.reconstructible` and fails closed – blocks the action – when any of the following is true: - high-impact action by agent is not reconstructible from immutable logs

- **Maturity target (L3):** Immutable logs with integrity verification. Full session reconstruction capability. Multi-agent chain reconstruction.
- **Incident evidence (6):** INC-07, INC-40, INC-46, INC-53, INC-70, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

GRM — Governance & Risk

GRM-01: AI governance body

- **Executable policy:** `shield_ai_grm_grm-01.rego` (package `shield_ai.grm.ai_governance_body`, 3 deny rules)

What to verify: AI governance body must hold. Establish and charter an AI governance body with cross-functional membership (CISO, CRO, GC, data owner, AI lead) and defined decision authority over agentic AI deployments. The policy evaluates `input.governance.body = {"chartered": bool, "cross_functional":` and fails closed – blocks the action – when any of the following is true: - deployment without a chartered AI governance body - AI governance body lacks cross-functional membership (CISO, CRO, GC, data owner, AI lead) - AI governance body has no defined decision authority over agentic AI deployments

- **Incident evidence (1):** INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

GRM-02: AI risk register

- **Executable policy:** `shield_ai_grm_grm-02.rego` (package `shield_ai.grm.ai_risk_register`, 2 deny rules)

What to verify: AI risk register must hold. Maintain a dedicated AI risk register using a threat taxonomy that covers model, tool, data, identity, and pipeline risks, with defined owners, assessment methodology, and review cadence. The policy evaluates `input.governance.risk = {"registered": bool,` and fails closed – blocks the action – when any of the following is true: - kill-switch capability not exercised (AI-speed response requires one rehearsed red button) - … action not registered in the AI risk register

- **Incident evidence (6):** INC-06, INC-32, INC-35, INC-49, INC-55, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

GRM-03: Ethical use policy

- **Executable policy:** `shield_ai_grm_grm-03.rego` (package `shield_ai.grm.ethical_use_policy`, 2 deny rules)

What to verify: Ethical use policy must hold. Document and enforce an ethical use policy that classifies agents by risk tier, defines autonomy boundaries per tier, and specifies transparency obligations. The policy evaluates `input.action.acceptable (bool) -- the action is inside the` and fails closed – blocks the action – when any of the following is true: - high-risk tier agent deployed without human-in-the-loop requirement per risk tier - action outside the documented acceptable-use boundary for agent

- **Incident evidence (3):** INC-10, INC-32, INC-42 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

GRM-04: Third-party AI risk management

- **Executable policy:** `shield_ai_grm_grm-04.rego` (package `shield_ai.grm.third_party_ai_risk_management`, 3 deny rules)

What to verify: Third-party AI risk management must hold. Assess and monitor third-party AI providers for security controls, with contractual security requirements and right-to-audit provisions in all AI-related procurement. The policy evaluates `input.vendors = {"assessed": bool, "contractual_controls": bool,` and fails closed – blocks the action – when any of the following is true: - action using third-party vendor without security assessment - vendor without contractual security control obligations - evaluation vendor without right-to-audit (harness egress must be independently verified, never self-attested)

- **Incident evidence (6):** INC-26, INC-62, INC-63, INC-75, INC-90, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

GRM-05: Regulatory mapping and compliance reporting

- **Executable policy:** `shield_ai_grm_grm-05.rego` (package `shield_ai.grm.regulatory_mapping_and_complia`, 2 deny rules)

What to verify: Regulatory mapping and compliance reporting must hold. Maintain an automated mapping of SHIELD-AI controls to applicable regulatory frameworks with continuous compliance monitoring and regular reporting. The policy evaluates `input.governance.compliance = {"mapped": bool,` and fails closed – blocks the action – when any of the following is true: - deployment without an automated regulatory mapping (ISM, E8, SOCI, CPS 234, PSPF, Privacy Act) - compliance posture not reported on the quarterly cadence

- **Incident evidence (1):** INC-16 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

GRM-06: Board and executive reporting

- **Executable policy:** `shield_ai_grm_grm-06.rego` (package `shield_ai.grm.board_and_executive_reporting`, 2 deny rules)

What to verify: Board and executive reporting must hold. Provide a standardised executive dashboard showing pillar maturity scores, open findings, regulatory compliance status, and risk acceptances, refreshed automatically. The policy evaluates `input.governance.reporting = {"dashboard": bool,` and fails closed – blocks the action – when any of the following is true: - no executive dashboard for AI security posture (maturity, findings, compliance, risk acceptances) - risk acceptance decision not logged in the executive reporting trail

- **Incident evidence (3):** INC-73, INC-90, INC-92 – opens each incident's human-readable summary (see doc 07)

→ **Deeper guide, examples & what to look for** — worked exploit/incident examples and what a security architect should check in a typical deployment.

Verification protocol

- Run `make test` — the OPA suite evaluates every `policies/*.rego` (69 files, 195 deny rules).
- A control is met when its policy's `deny` set is empty for the input under test.
- Missing input fields are treated as violations (fail closed).